Übungen – Arbeiten mit dem Coding-Agenten auf dem gPlug

Diese Übung als PDF (A4) · Begriffe: LEGENDE.md

Gegenstand der Übungen ist der **Arbeitszyklus mit dem Agenten**: Kontext geben → kleine, prüfbare Aufgabe formulieren → Code aufs Gerät laden → Konsole lesen → Fehler wörtlich zurückgeben. Die Berry-App `app/spot.be` ist nur das Übungsobjekt. Die Übungen ändern nur die Darstellung, nie den Datenfluss; bei Fehlern bleibt der Kern der App intakt.

Grundlagen: `AGENTS.md` (Regeln und Werkzeuge, liest der Agent automatisch), `TASMOTA_BERRY_API.md` (die Tasmota-API, ohne die der Agent rät), `SETUP_OPENCODE.md` §7 (Arbeitsweise im Agenten).

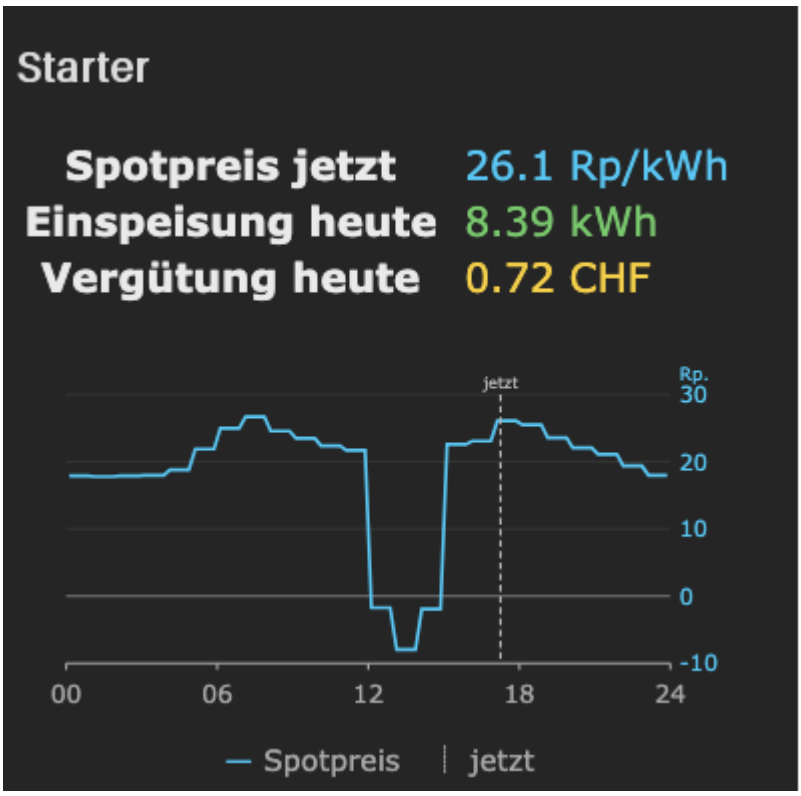
Überblick: vom Starter zur Referenz-App in 4 Schritten

Teilnehmende erhalten im Docker-Container einen **Starter** und bauen die Grafik in vier Schritten bis zum Zielbild `docs/zielbild.jpg` aus (entschieden 22.10.).

Schritt	Ziel	Sichtbares Resultat	Lernziel KI
0 Starter	–	Preislinie, 1 Tag, ¼ h	–
1 Energie-Balken	Einspeisung pro ¼ h als grüne Balken, eigene kWh-Achse links	Solarkurve unter der Preislinie	Kontext geben: Erweiterungspunkt und Datenquelle nennen
2 Negativpreis	Balken orange bei Preis < 0, Bereich unter 0 Rp. schattiert, Legende	Mittagstief fällt auf	Kleine, prüfbare Aufgabe; Testfall benennen (Sim: Mittagstief < 0)
3 3 Tage, Stundenwerte	x-Achse 72 h, Tagesnamen, 00/12-Marken; Stunde = Mittel (Preis) bzw. Summe (Energie)	Do–Sa wie im Zielbild	Umbauen statt anbauen; Heap vorher/nachher messen
4 Vergütungs-Chart	Zweites Chart darunter: CHF/h gelb/rot, Σ über 3 Tage	Zielbild komplett	Auf eigenem Ergebnis aufbauen; Wiederverwendung einfordern

- Jeder Schritt ist in der WebUI sichtbar und für sich lauffähig. 1–2 ändern nur den einen Tag (klein, schnell), 3 ist der Umbau, 4 baut auf 3 auf. Schritte 1–2 gelingen auch, wenn 3 scheitert.
- **Pflicht: Schritte 1 und 2. Schritte 3 und 4 für Schnelle.** Zeitbudget im Übungsblock 55 min.
- **Checkpoints**: `app/steps/spot_1.be` … `spot_4.be` enthalten den fertigen Stand nach jedem Schritt. Wer hängen bleibt, kopiert den vorherigen Checkpoint nach `app/spot.be` und macht dort weiter (`cp app/steps/spot_2.be app/spot.be && ./upload.sh app/spot.be`).

Starter (Schritt 0)



Daten vollständig, Darstellung minimal:

- Im Hintergrund: Preisabruf (F1), Zähler mit Simulator inkl. Zeitraffer, `Po` und gefüllten Vortagen (F2), Tages-CSV (F4), `SpotPrices` (F5). Der Speicher hält **3 Tage** Preise und Energie.
- Textzeilen über dem Chart: Spotpreis jetzt, Einspeisung heute, Vergütung heute. Damit sind die Daten sichtbar, bevor sie gezeichnet werden; Schritt 1 lässt sich an «Einspeisung heute» prüfen.
- Chart: **nur die Preislinie** (cyan, `COL_PRICE`), 1 Tag mit 96 ¼-h-Werten, Achse Rp./kWh rechts, Nulllinie, «jetzt»-Marke (grau), Legende aus `chart_curves()`. Farben wie im Zielbild; `COL_ENERGY` (grün), `COL_NEG` (orange), `COL_PAY` (gelb), `COL_COST` (rot) sind schon definiert.
- Simulator gibt dem heutigen Tag ein Mittagstief mit negativen Preisen (Testfall Schritt 2).
- Vorhandene Helfer: `chart_curves()`, `send_polyline(...)`, `value_range(...)`, `nice_range(...)` (runde Achsenstufen), `y_scale(...)`, `send_data_attr(...)` (Werte für den Cursor), Farbkonstanten `COL_...`, Datenzugriff `self.store.price(key, slot)` (1/10'000 CHF/kWh) und `self.store.energy_at(key, slot)` (0.01 kWh), beide `nil` = Lücke.
- Kommentare `# ERWEITERUNGSPUNKT Schritt 1` … `Schritt 4` am Anfang von `F3  Chart`; jeder Checkpoint enthält nur noch die Punkte der offenen Schritte.
- Cursor-Script in allen Stufen gleich: es zeigt jeden vorhandenen Wert (`data-p`, `data-e`, `data-v`) und rechnet die Zeit aus `data-m` (15 oder 60 min). Kein Schritt ändert das Script.

Vorbereitung (5 min)

1. Geräte-IP prüfen (eingetragen im RUNBOOK, Schritt 6.1). Erwartet: die IP, z.B. `192.168.1.50`.

   ```
   cat .gplug
   ```

2. Gerät antwortet, erwartet JSON mit `"Sim":true`. **`HeapFree` notieren** (Ausgangswert, z.B. 125).

```
./brcmd.sh 'tasmota.cmd("SpotStatus")'
```

3. WebUI des gPlug im Browser offen lassen (`http://192.168.x.y`).
4. opencode starten:

   ```
   ./docker-run.sh
   ```

   Nur ohne Docker (nativ installiert, `SETUP_OPENCODE.md`), im Repo-Ordner:

   ```
   opencode
   ```

   opencode startet im **build**-Agent; vor dem ersten Prompt auf **plan** schalten (`Tab`, siehe *Ablauf pro Schritt*).

Ablauf pro Schritt

opencode hat zwei Agenten. Welcher aktiv ist, steht unten im Eingabefeld (`Plan` bzw. `Build`). Umschalten mit **`Tab`** (oder `Shift+Tab`), jedes Drücken wechselt zum anderen Agenten. Der Gesprächsverlauf bleibt beim Wechsel erhalten: der Build-Agent kennt den Plan.

Agent	darf	wofür
plan	nur lesen, nichts ändern, nicht hochladen	Prompt «planen»
build	Dateien ändern, Befehle ausführen (nach Freigabe)	Prompt «umsetzen», «Fehler zurückgeben»

opencode startet im **build**-Agent. Jeder Schritt läuft so:

1. **Auf plan schalten**: `Tab` drücken, bis unten `Plan` steht.
2. Prompt «planen» des Schritts einfügen, absenden. Plan lesen; weicht er von den Vorgaben ab (eigene Zeichenschleife, Checkboxen, JavaScript): im plan-Agent ablehnen, Vorgabe zitieren, neuen Plan verlangen. Erst weiter, wenn der Plan stimmt.
3. **Auf build schalten**: `Tab` drücken, bis unten `Build` steht.
4. Prompt «umsetzen» (gleich für alle Schritte, unten) einfügen, absenden. Shell-Befehle einzeln freigeben (nur `./upload.sh`, `./log.sh`, `./brcmd.sh`, `berry`, `cp`, `git`).
5. Fehler im **build**-Agent mit Prompt «Fehler zurückgeben» melden (nicht umschalten).
6. Prüfschritte selbst durchgehen, in der WebUI ansehen, Stand sichern: Kopie `cp app/spot.be spot_schritt1.be` (Nummer des Schritts); wer git kennt: `git commit`.
7. Nächster Schritt: wieder bei 1, **zurück auf plan**.

Merkregel: **planen = plan, alles andere = build.** Antwortet der Agent auf «umsetzen» nur mit Text und ändert nichts, ist noch plan aktiv: auf build schalten, Prompt nochmals senden.

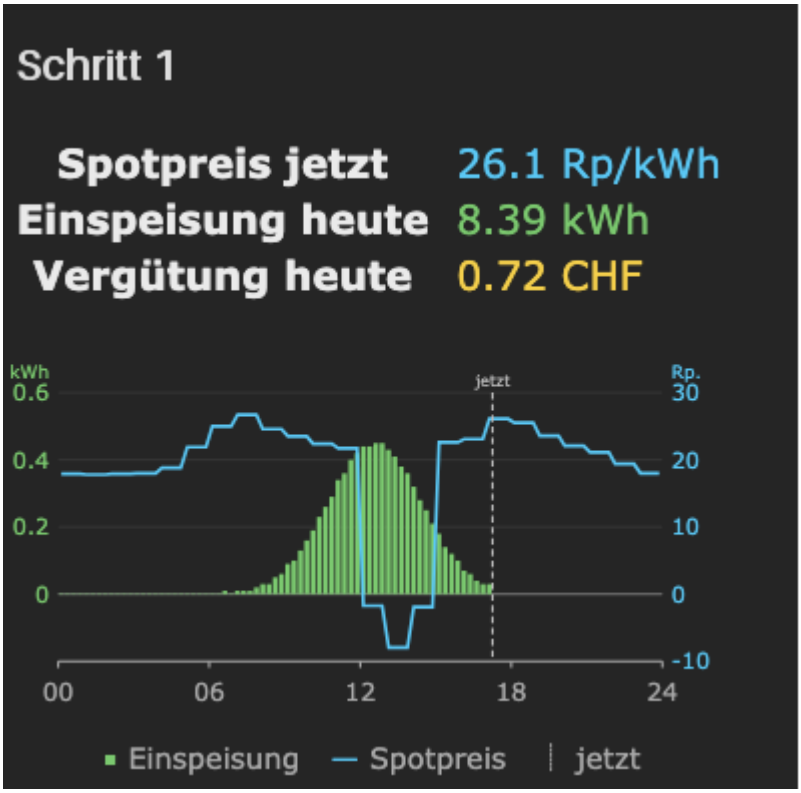
Prompt «umsetzen» (alle Schritte, im build-Agent)

```
Setze den Plan um, in dieser Reihenfolge:
1. Änderung in app/spot.be.
2. cd app && berry desktop_harness.be — muss mit HARNESS OK enden.
3. ./upload.sh app/spot.be, danach ./log.sh. Jede Zeile mit BRY: oder SPOT: wörtlich zeigen.
4. ./brcmd.sh 'tasmota.gc()' und ./brcmd.sh 'tasmota.memory("heap_free")', Heap melden.
Melde in fünf Zeilen: was geändert, was geprüft, was offen. Die WebUI prüfe ich selbst.
```

Prompt «Fehler zurückgeben» (im build-Agent)

```
./log.sh zeigt nach dem Upload:
<Zeilen wörtlich einfügen>
Behebe nur diesen Fehler, lade neu hoch und lies das Log erneut.
```

Schritt 1 – Energie-Balken (Pflicht, ~20 min)



Aufgabe

Das Chart zeigt nur die Preislinie. Ergänze die **eingespeiste Energie pro Viertelstunde (kWh) als grüne Balken** hinter der Linie, mit eigener Achse links.

Vorgaben:

- Datenquelle `self.store.energy_at(key, slot)` (Einheit 0.01 kWh, `nil` = Lücke, kein Balken).
- Kurzbuchstabe `e` (SVG-Gruppe `se`), Farbe `COL_ENERGY` (grün).
- Achse **links** mit kWh-Werten von 0 bis Tagesmaximum, Beschriftung «kWh» oben links. Das ist die einzige neue Achse; die Preisachse rechts bleibt unverändert.
- Eintrag «Einspeisung» in der Legende mit Balkensymbol `■`, ein-/ausblendbar wie Spotpreis.
- Balken vor der Preislinie zeichnen (Linie liegt oben).
- Bestehende Helfer verwenden (`chart_curves`, `value_range`, `y_scale`). Keine neue Schleife über die Slots ausserhalb einer Methode `send_bars`, kein JavaScript, nichts anderes ändern.

Prompt «planen» (im plan-Agent, `Tab` bis unten `Plan` steht)

```
Lies @AGENTS.md, @TASMOTA_BERRY_API.md und in @app/spot.be den Abschnitt "F3  Chart" mit dem
Kommentar "ERWEITERUNGSPUNKT Schritt 1".

Aufgabe: Im Tageschart soll die eingespeiste Energie pro Viertelstunde (kWh) als grüne Balken
hinter der Preislinie erscheinen.

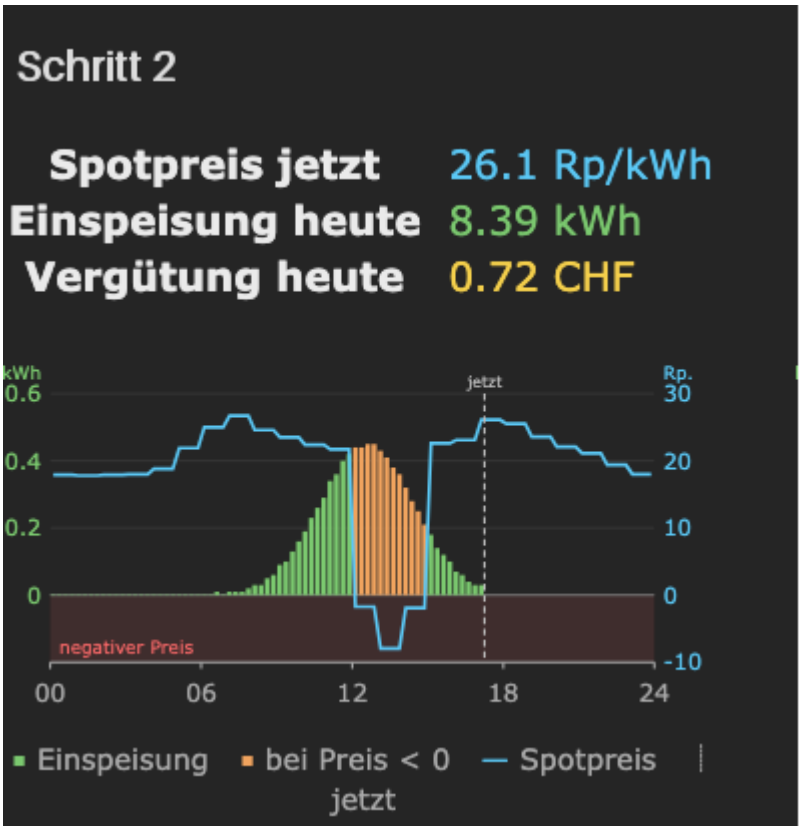
Vorgaben: Datenquelle self.store.energy_at(key, slot) (0.01 kWh, nil = Lücke). Kurzbuchstabe e,
Farbe COL_ENERGY. Eine neue Achse links mit kWh von 0 bis Tagesmaximum (ausdrücklich erlaubt),
Preisachse rechts unverändert. Legendeneintrag "Einspeisung" mit &#9632;, ein-/ausblendbar.
Bestehende Helfer verwenden: chart_curves, value_range, y_scale. Kein JavaScript, nichts anderes
ändern.

Schreibe einen Plan: welche Methoden du änderst oder neu anlegst und wie du das Ergebnis prüfst
(Desktop-Harness, upload.sh, log.sh, Heap). Frag nach, wenn etwas unklar ist. Noch nichts ändern.
```

Erwartetes Resultat

- Grüne Balken mit Tageskurve (Simulator: Maximum um 12:40, Sockel 0.3 kW = 0.075 kWh/¼ h).
- Achse links in kWh, Preisachse rechts unverändert, Legende mit «Einspeisung».
- Summe der Balken ≈ «Einspeisung heute» in der Kopfzeile.
- `app/sensor.html` (Harness) enthält `<g id='se'>`.

Schritt 2 – Negativpreis hervorheben (Pflicht, ~15 min)



Aufgabe

Einspeisung bei negativem Preis kostet Geld. Mach das sichtbar: **Balken orange, wenn der Preis im Slot < 0 ist**, und **Bereich unter 0 Rp. schattiert** mit Text «negativer Preis».

Vorgaben:

- Farbe `COL_NEG` (orange) für Balken mit `self.store.price(key, slot) < 0`, sonst `COL_ENERGY`.
- Schattierung: ein Rechteck von der Nulllinie bis zum unteren Rand, halbtransparent, hinter allem. Nur zeichnen, wenn der Tag negative Preise hat (die Preisachse reicht dann unter 0).
- Legendeneintrag «Einspeisung bei Preis < 0» in Orange.
- Testfall: Im Sim-Modus hat der heutige Tag ein Mittagstief mit negativen Preisen (Preislinie im Chart unter 0). Nicht über `SpotPrices` prüfen: das zeigt nur die nächsten 10 h, am Abend ist das Mittagstief vorbei.

Prompt «planen» (im plan-Agent, `Tab` bis unten `Plan` steht)

```
Lies in @app/spot.be den Abschnitt "F3  Chart" mit dem Kommentar "ERWEITERUNGSPUNKT Schritt 2".

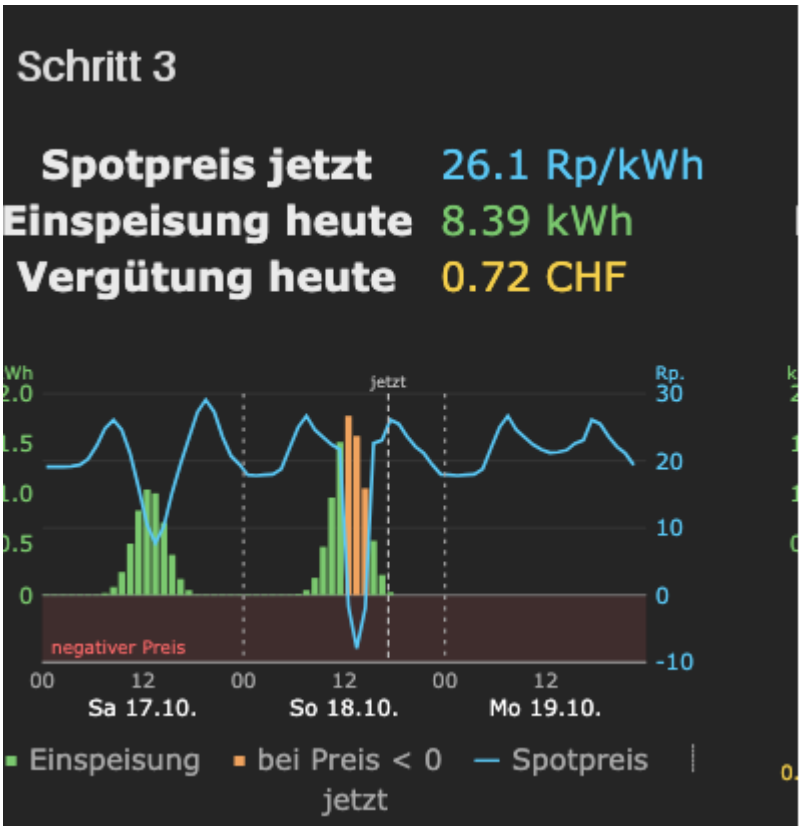
Aufgabe: Energie-Balken in Slots mit negativem Preis orange (COL_NEG) statt grün. Bereich unter
der Nulllinie halbtransparent schattieren mit Text "negativer Preis", nur wenn der Tag negative
Preise hat. Legendeneintrag "Einspeisung bei Preis < 0" in Orange.

Nichts anderes ändern. Nenne im Plan, wie du prüfst, dass orange Balken genau bei negativem
Preis erscheinen (der Simulator liefert heute ein Mittagstief < 0). Noch nichts ändern.
```

Erwartetes Resultat

- Orange Balken genau dort, wo die Preislinie unter 0 liegt (Sim: Mittagstief).
- Schattierung nur, wenn der Tag negative Preise hat; ohne sieht das Chart aus wie nach Schritt 1.

Schritt 3 – 3 Tage mit Stundenwerten (für Schnelle, ~20 min)



Aufgabe

Das Chart zeigt **die 3 Tage im Speicher** (Ringpuffer, ältester links; ab ~17 Uhr ist der jüngste Tag morgen) **mit Stundenwerten** statt einem Tag mit Viertelstunden.

Vorgaben:

- 72 Werte. Stundenwert: Preis = **Mittel**, Energie = **Summe** der 4 Viertelstunden. Fehlen alle 4 Werte: Lücke. Stundenwerte **einmal pro Aufruf** von `page_svg` (Handler `/spot.svg`) rechnen (`fill_hours(start)` in feste `bytes`-Puffer), Kurven lesen sie mit `hour_p(i)`/`hour_e(i)`. Grund: 72 × 4 Store-Zugriffe pro Kurve überschreiten sonst die 50-ms-Grenze.
- x-Achse: Tagesgrenzen gestrichelt, Marken 00 und 12, Tagesname mit Datum unter jedem Tag («Do 11.06.»).
- «jetzt»-Marke an der richtigen Stunde über alle 3 Tage.
- Orange-Regel aus Schritt 2 gilt pro Stunde (Stundenpreis < 0).
- Bestehende Methoden umbauen, nicht kopieren: `value_range`, `send_polyline`, `send_bars`, `send_x_axis` bekommen den Bereich (Starttag, Anzahl Werte) als Parameter.
- **Heap**: vor und nach der Änderung `tasmota.gc()` + `heap_free` messen, Differenz melden. Mehr als 5 kB Einbruch beim Rendern: anhalten.

Prompt «planen» (im plan-Agent, `Tab` bis unten `Plan` steht)

```
Lies in @app/spot.be den Abschnitt "F3  Chart" mit dem Kommentar "ERWEITERUNGSPUNKT Schritt 3".

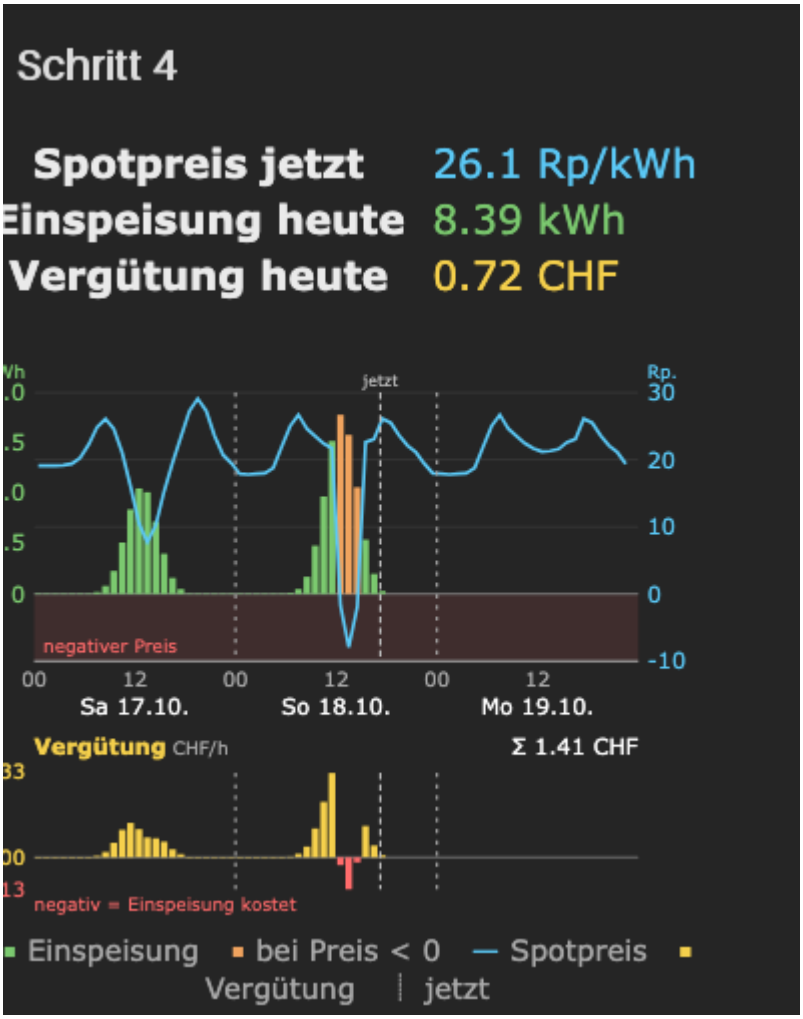
Aufgabe: Das Chart zeigt 3 Tage mit 72 Stundenwerten statt einem Tag mit 96 Viertelstunden.
Stundenwert: Preis = Mittel, Energie = Summe der 4 Viertelstunden, Lücke wenn alle 4 fehlen.
Stundenwerte einmal pro page_svg-Aufruf in feste bytes-Puffer rechnen (fill_hours), nicht pro Kurve.
x-Achse mit gestrichelten Tagesgrenzen, Marken 00 und 12, Tagesname mit Datum ("Do 11.06.").
"jetzt"-Marke über alle 3 Tage richtig. Orange-Regel aus Schritt 2 pro Stunde.

Baue die bestehenden Methoden um (Bereich als Parameter), kopiere sie nicht. Miss den Heap vor
und nach der Änderung (tasmota.gc(), dann heap_free). Liste im Plan jede Methode auf, die du
änderst, und warum. Noch nichts ändern.
```

Erwartetes Resultat

- 3 Tage nebeneinander, Tagesnamen, 72 Balken, Preislinie in Stundenstufen.
- Vergleich mit der Sicherung des vorherigen Schritts: Umbau in bestehenden Methoden, keine kopierten `send_..._3d`-Varianten.
- Heap nach Upload und `tasmota.gc()` höchstens ~2 kB unter dem Wert vor dem Schritt.

Schritt 4 – Vergütungs-Chart (für Schnelle, ~15 min)



Aufgabe

Unter dem bestehenden Chart ein **zweites, flaches Chart: Vergütung pro Stunde (CHF)**, gleiche x-Achse, mit **Summe Σ über die 3 Tage** oben rechts.

Vorgaben:

- Vergütung pro Stunde = Summe über die 4 Viertelstunden von Preis × Energie. In `fill_hours` aus Schritt 3 mitrechnen (dritter Puffer, `hour_v(i)`), nicht neu schleifen.
- Balken gelb (`COL_PAY`) bei ≥ 0, rot (`COL_COST`) bei < 0. Nulllinie, Achse links mit Max/0/Min in CHF. Text «negativ = Einspeisung kostet» unten links, nur wenn es negative Werte gibt.
- Titel «Vergütung CHF/h» links, «Σ x.xx CHF» rechts.
- Gleiche x-Positionen wie oben (Stunde n liegt in beiden Charts übereinander), «jetzt»-Marke auch unten.
- Stückweise senden wie das obere Chart (`tasmota.web_send` pro Teil), kein Gesamtstring.

Prompt «planen» (im plan-Agent, `Tab` bis unten `Plan` steht)

```
Lies in @app/spot.be den Abschnitt "F3  Chart" mit dem Kommentar "ERWEITERUNGSPUNKT Schritt 4".

Aufgabe: Unter dem Chart ein zweites, flaches Chart "Vergütung CHF/h": pro Stunde Summe aus
Preis × Energie der 4 Viertelstunden, Balken gelb (COL_PAY) bei >= 0, rot (COL_COST) bei < 0,
Achse links in CHF, Summe "Σ x.xx CHF" oben rechts über die 3 Tage. Gleiche x-Positionen wie
oben, "jetzt"-Marke auch unten.

Verwende die Stundenwert-Hilfsmethode aus Schritt 3 und die bestehenden Zeichenmethoden weiter.
Sende stückweise mit tasmota.web_send. Nenne im Plan, welche Methoden du wiederverwendest und
welche neu sind. Noch nichts ändern.
```

Erwartetes Resultat

- Zielbild `docs/zielbild.jpg` erreicht: zwei Charts, Stunden übereinander, Σ stimmt mit der Summe der Tageswerte überein (Kopfzeile «Vergütung heute» + Vortage).
- Heap nach Upload und `tasmota.gc()` > 100 kB, Rendering ohne Einbruch > 5 kB.

Prüfschritte (alle Schritte)

```
cd app && berry desktop_harness.be | tail -1      # HARNESS OK
cd ..
./upload.sh app/spot.be                           # OK ... heap_free N kB
./log.sh                                          # keine BRY:-Zeile
./brcmd.sh 'tasmota.cmd("SpotStatus")'            # Days vorhanden, HeapFree nahe Ausgangswert
./brcmd.sh 'tasmota.gc()'; ./brcmd.sh 'tasmota.memory("heap_free")'
diff spot_schritt2.be app/spot.be | head -40      # wenige Stellen (oder git diff --stat)
```

Im Browser: Seite **neu laden** (Cmd+R / F5), Kurven per Legende aus- und einblenden, mit der Maus Werte anfahren.

Stolpersteine (aus dem Probelauf 19.10., 30 min, 14 Uploads)

1. **Weitere Achsen.** Erlaubt ist nur die kWh-Achse links aus Schritt 1 (und die CHF-Achse im eigenen Chart aus Schritt 4), so steht es auch in `AGENTS.md`. Will der Agent mehr beschriften: nein sagen, Vorgabe zitieren.
2. **Checkboxen / `onchange` im Chart** funktionieren nie: Tasmota ersetzt den Sensorbereich alle paar Sekunden per AJAX, Scripts darin laufen nicht. Legende über `chart_curves()` erweitern.
3. **Eigene Zeichenschleife.** Der Agent kopiert die Polyline-Logik (gern zweimal). `send_polyline` bzw. `send_bars` einfordern.
4. **Kopieren statt umbauen** (Schritt 3/4): `send_x_axis_3d`, `value_range2` usw. Prompt: «Bau die bestehende Methode um, lösche die Kopie.»
5. **Heap.** Jeder Upload kostet ~1 kB, den erst ein Neustart zurückgibt; `upload.sh` startet das Gerät unter 100 kB selbst neu. Scheitert der Agent zweimal gleich: anhalten, Log selbst lesen.
6. **Erfundene API** (`webserver.content_send` in `web_sensor`, nicht existierende `tasmota.*`-Funktionen). Auf `TASMOTA_BERRY_API.md` verweisen; nur dort Genanntes gilt.
7. **`BrRestart` / `Restart` über `brcmd.sh`** sind gesperrt (Absturz bzw. Reboot). App neu laden nur über `./upload.sh`.
8. **Fehlendes `var`** für eine neue Instanzvariable → `BRY:`-Fehler mit Zeilennummer beim Laden (strict mode). Zeile wörtlich an den Agenten.
9. **Scope-Creep** («bei der Gelegenheit habe ich auch …»): «nichts anderes ändern» wiederholen, Änderung mit `/undo` zurücknehmen.

10. **Stubs aufs Gerät** (`persist.be`, `path.be`, `webserver.be`, `desktop_harness.be`): `upload.sh` lehnt ab. Der Agent soll nur `app/spot.be` hochladen.
11. **Chart oder Cursor fehlt** nach einem Upload: Die Seite läuft noch mit dem alten Script, das `/spot.svg` holt und den Cursor zeichnet (Tasmota ersetzt per AJAX nur den Sensorbereich, das Script lädt nur beim Seitenaufruf). Seite neu laden, kein Fehler im Code. Das Chart steht nicht im Sensorbereich, sondern kommt vom Handler `/spot.svg` (`page_svg`); `web_sensor` sendet nur Zahlen und den Platzhalter `#spotchart`. Will der Agent das Chart zurück in `web_sensor` holen: nein (kostete 1.2 s pro Refresh, Gerät stürzte ab).
12. **Shell-Freigaben.** opencode fragt vor jedem Befehl. Nur `./upload.sh`, `./log.sh`, `./brcmd.sh`, `berry`, `cp`, `git` freigeben; alles andere ablehnen und den Agenten fragen, wozu.

Bonus (wer fertig ist)

- **CSV mit Stundenwerten**: zusätzlich zur Tageszeile `spot_h_YYYY-MM.csv`, Aufbewahrung zwei Monate. Erweiterungspunkt `cfg["csv_hourly"]` und `day_end()`. Lehrt: Testbarkeit einfordern (Funktion mit Datums-Parameter verlangen).
- **Schwellwert an Shelly/MQTT**: bei Preis unter/über Schwelle ein HTTP-schaltbares Gerät schalten oder per `mqtt.publish` melden. Erweiterungspunkt `json_append()` / neuer Command. Lehrt: Drittsystem-API als Kontext geben (Shelly-Doku-Auszug), sonst rät der Agent.

Checkliste Abschlussrunde

- Der Prompt, der funktioniert hat (wörtlich).
- Der Prompt, der schiefging, und was der Agent daraus gemacht hat.
- Anzahl Upload-Zyklen und erreichter Schritt.
- `HeapFree` am Anfang und am Ende.